AgentSouq – Agentic Commerce on Arc

The Stablecoin Commerce Stack Challenge · **Track 4: Best Agentic Economy Experience on Arc**

Live MVP	`https://agentsouq.vercel.app`
Code & docs	`https://github.com/blinks-labs/agentsouq`
Video demo	`https://youtu.be/oTlzQ7vSdOY`
Circle account	nabsarkar@gmail.com
Circle products	USDC (settlement + gas, EIP-3009) · Circle Wallets (developer-controlled) · Nanopayments (concept) · Gateway/CCTP (roadmap)

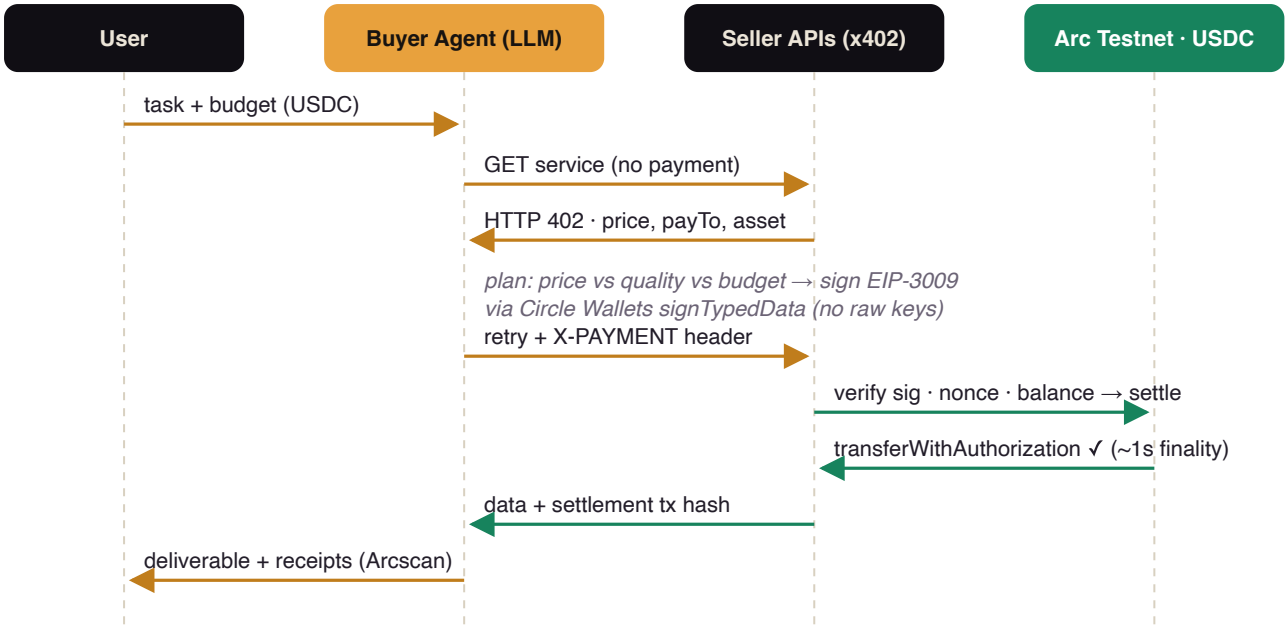
What it is

AgentSouq is a marketplace ("souq") where **AI agents are the customers**. A buyer agent receives a task and a hard USDC budget, discovers priced services via **x402** (HTTP 402) payment challenges, reasons about which counterparties to buy from – weighing price against quality and latency across competing sellers – signs gasless **EIP-3009** USDC authorizations with its **Circle developer-controlled wallet**, and every purchase settles on-chain on **Arc testnet** in seconds. It demonstrates autonomous stablecoin-settled purchases, multi-counterparty negotiation, pay-per-call (nanopayment-scale) pricing, and budget-governed agent spending – the core primitives of the agentic economy. (Educational / testnet demo only.)

Why it matters for the agentic economy

- **Agents negotiate with counterparties:** two FX providers offer the same category at different price/quality points; in live runs the LLM planner paid 2.5× more for the premium provider when the task demanded real-time pricing, and picked the budget provider when it didn't – and explains each choice.
- **No raw keys anywhere:** the agent's wallet is a Circle developer-controlled wallet; EIP-3009 typed data is signed via Circle's `signTypedData` API. The buyer never holds a private key and never pays gas.
- **One-asset economics:** on Arc, gas *is* USDC – sellers' settler pays cents of USDC gas, removing the classic "agent needs two tokens" bootstrapping problem.
- **Cent-level pricing:** services cost $0.02–$0.10 per call; the identical flow supports sub-cent authorizations for high-frequency agentic payments.
- **Budget governance:** the agent enforces a hard cap in code, reports spend, and explains skipped purchases.

Architecture



Single Next.js app: buyer agent (`src/lib/agent.ts`, DeepSeek via OpenRouter), Circle Wallets signer (`src/lib/circle.ts`), x402 verify/settle engine (`src/lib/payments.ts`), four paid seller endpoints across three sellers, and a chat + artifacts dashboard. Five wallets on `ARC-TESTNET`: agent (Circle), 3 seller treasuries (Circle), settler EOA (pays USDC gas).

Live on-chain proof (Arc testnet)

Purchase	Amount	Settlement tx (testnet.arcscan.app)
FastFX real-time FX quotes	`$0.05`	`0x95fadffd20d77bb0ba869162aea5bc59402f232c5618612bde52d3ec5031648b`
SouqData corridor market brief	`$0.10`	`0x8b772abc9cd6f4afacb4f4ecf2886e5fabee12de42f1b9dd6652400a52230a9b`
LingoPay translation (TL/HI/AR)	`$0.03`	`0x0a29e2c1616edcfc12fd3b3ba1b6a2a63e4120918a7795a3d08bfcd41c5265e1`

Every dispatched run produces fresh settlements — the dashboard links each payment to its transaction. Access to the public demo is wallet-gated (any EVM wallet, signature only — no funds or gas) with per-wallet rate limits and a $0.25 hard cap per run.

How the x402 payment works

- Seller replies **402** with an x402 `accepts` array: scheme `exact`, network `eip155:5042002`, price, `payTo`, asset (Arc-native USDC `0x3600…0000`, FiatToken v2).
- Agent builds an EIP-3009 `TransferWithAuthorization` (random 32-byte nonce, 5-min validity) and signs it through Circle Wallets.
- Seller verifies signature, unused nonce (`authorizationState`), balance, amount, and window — then the settler submits `transferWithAuthorization` on-chain and serves the response with the tx hash (`X-PAYMENT-RESPONSE`).

Documentation & setup

Full documentation lives in the repository README: environment setup, one-command Circle wallet provisioning scripts (`circle-setup.mjs` registers the entity secret and creates the wallet set + agent wallet on `ARC-TESTNET`; `create-wallets.mjs` creates seller treasuries), faucet funding, the x402 flow in detail, and the architecture diagram in mermaid. Run locally with `pnpm install && pnpm dev` after the setup scripts.

Circle Product Feedback

Why these products: we wanted an agent that holds no raw keys — Circle Wallets custody plus `signTypedData` maps perfectly onto EIP-3009 payment authorizations — and a chain where the settlement asset is also the gas asset (Arc), which removes the "agent needs two tokens" bootstrapping problem.

What worked well:

- Wallet creation on `ARC-TESTNET` worked first try via the SDK; entity-secret registration is fully scriptable.
- `signTypedData` on a developer-controlled wallet signed EIP-3009 typed data that verified on-chain with no surprises — Circle Wallets is a drop-in signer for x402/agentic payments.
- Arc's USDC keeps FiatToken v2's EIP-3009 intact at a well-known address, making gasless pull-payments trivial; USDC-as-gas collapses treasury logic to a single asset.
- Deterministic finality: `waitForTransactionReceipt` returns in ~1–2s, so payment-before-response APIs feel synchronous.

What could be improved:

- The faucet API (`POST /v1/faucet/drips`) returned `Forbidden` for a standard testnet key, forcing manual browser funding — programmatic faucet access for Arc would make agent demos fully scriptable.
- `registerEntitySecretCiphertext`'s `recoveryFileDownloadPath` expects a directory but errors confusingly when given a file path.
- No x402 facilitator or network-registry entry for Arc yet — we implemented exact-scheme verify/settle ourselves. A Circle-hosted facilitator for Arc would cut integration to minutes.
- A first-class "sign EIP-3009 payment" helper in the Wallets SDK (amount, payee, token → `X-PAYMENT` header) would make Circle Wallets the default wallet of the agentic economy.

Recommendations: ship an Arc x402 quickstart (facilitator + Wallets signer), expose faucet drips to standard testnet API keys, and add webhook events for inbound `transferWithAuthorization` credits so seller treasuries can react without polling.